

"The future of software development is paradoxical."

Voxxed Days Luxembourg 2026

Demystify SDD

Spec Driven Development

(when AI gives meaning back to specifications)

Long LE
Zied Ben Abderrahim

Qui sommes-nous?



Long LE

A T(S)DD Guy

Tech Lead @Bpifrance



Zied Abderrahim

Principial enginner **@Capco**

The problem with traditional spec?



Jira ticket

The exhaustive specification document, written once and never reopened.

The problem with traditional spec?



Jira ticket

The exhaustive specification document, written once and never reopened.



The 3-line Ticket

"Build feature X." Minimal context, relying purely on developer assumptions.

The problem with traditional spec?



Jira ticket

The exhaustive specification document, written once and never reopened.



The 3-line Ticket

"Build feature X." Minimal context, relying purely on developer assumptions.



Outdated wiki

"Actually, that's not what we wanted..."

The problem with traditional spec?



Jira ticket

The exhaustive specification document, written once and never reopened.



The 3-line Ticket

"Build feature X." Minimal context, relying purely on developer assumptions.



Outdated wiki

"Actually, that's not what we wanted..."

The spec lost its role as a contract. It became a dead artifact.





Vibe Coding?

Make a good prompt?

Pay for the best model AI (opus ?) !

Use all the best skills AI (skills.md) in the world ?



Vibe Coding

Make a good prompt

Pay for the best model AI (opus ?)

Use all the best skills AI in the world ?

AI agents are 10x faster at writing code than understanding intent. When intent is fuzzy, they produce confidently wrong code at machine speed.

"In 2026, the bottleneck is no longer typing code. The bottleneck is knowing what to build."

"How can we build (and keep controlling) with AI ?"

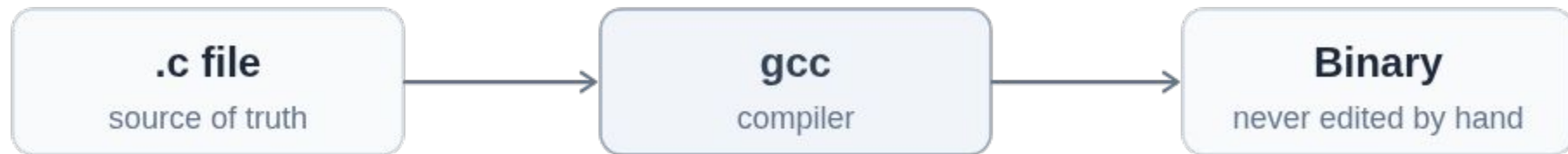
**Code is no longer the source of
truth. Intent is.**

— "The spec is the prompt" (GitHub,
2025)

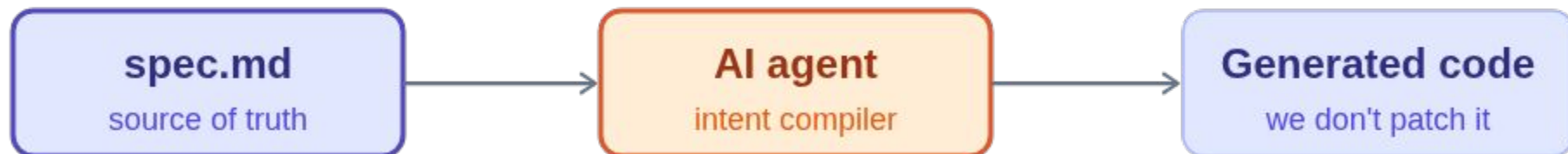
Spec-Driven Development (SDD)

A methodology where the specification is the primary artifact, and code is its compiled output.

The model we know



The same model, with AI



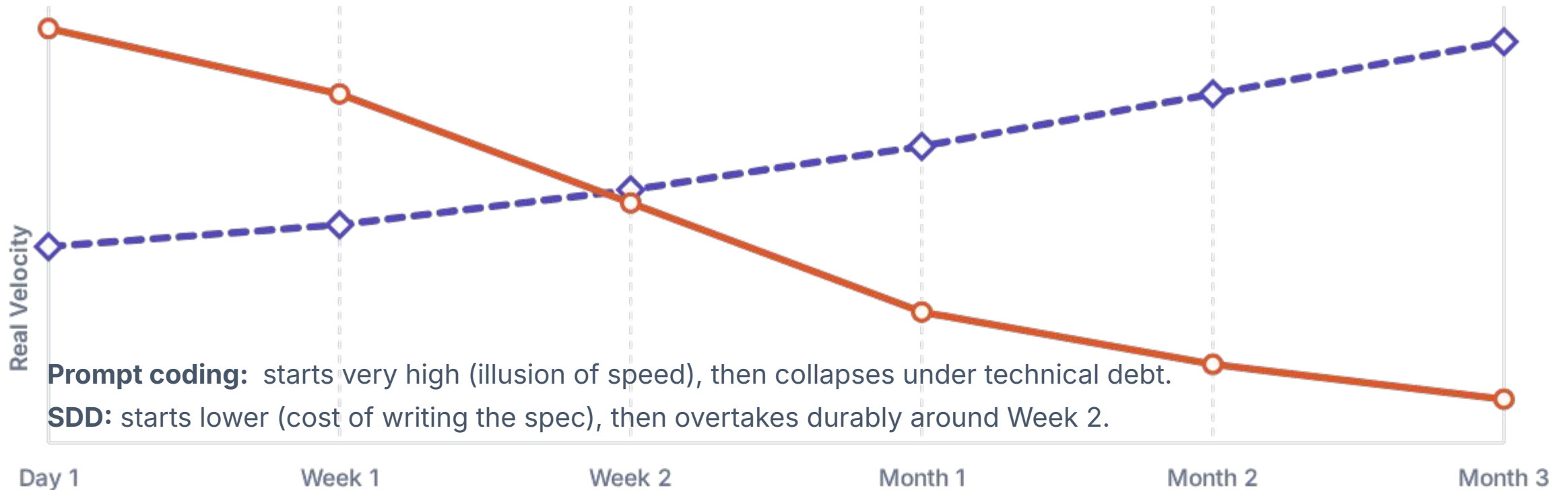
You change the intent, not the output — then you recompile

SDD vs. Traditional Practices

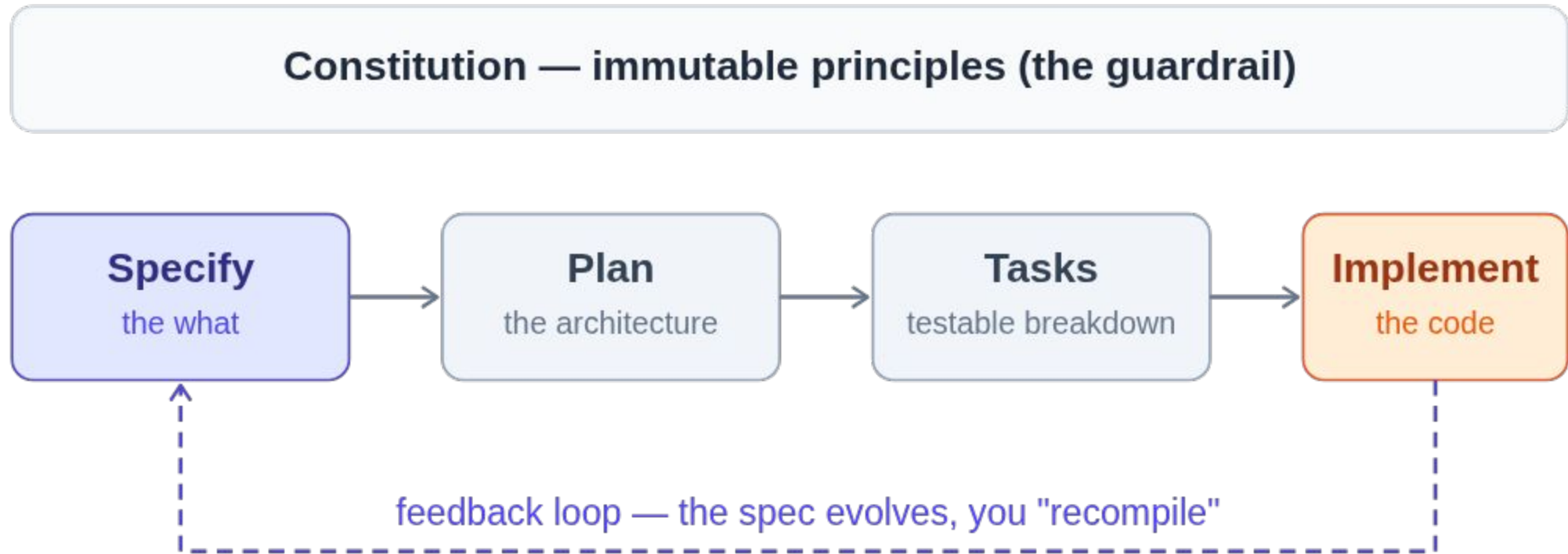
Practice	Source of Truth	Updated When?	AI Suitability
Code-first	The Code	During Implementation	Low (Gathers noise)
TDD	The Test	Before the Code	Medium (Unit scope)
BDD	Gherkin Scenarios	Before the Code	Medium (Business logic)
SDD	Structured Spec	Before Anything Else	High (Machine-ready)

Prompt coding vs SDD

— Prompt coding — SDD



How does it actually work?



The 2026 Ecosystem

- **GitHub Spec Kit:** Open source, 93k stars, model-agnostic → the default choice.
- **AWS Kiro:** Agentic IDE utilizing EARS notation.
- **Claude Code Skills / CLAUDE.md:** Lightweight and deeply integrated.
- **BMAD-METHOD:** 12+ specialized agents across the SDLC.
- **Emerging paradigms:** Cursor Plan Mode, OpenSpec, Tessl.

The 2026 Ecosystem

- **GitHub Spec Kit:** Open source, 93k stars, model-agnostic → the default choice.
- **AWS Kiro:** Agentic IDE utilizing EARS notation.
- **Claude Code Skills / CLAUDE.md:** Lightweight and deeply integrated.
- **BMAD-METHOD:** 12+ specialized agents across the SDLC.
- **Emerging paradigms:** Cursor Plan Mode, OpenSpec, Tessl.

"But today, we are using NONE of these tools. Because SDD isn't a question of tooling — it's a question of discipline. All you need: markdown files and an AI agent."

Live Demo

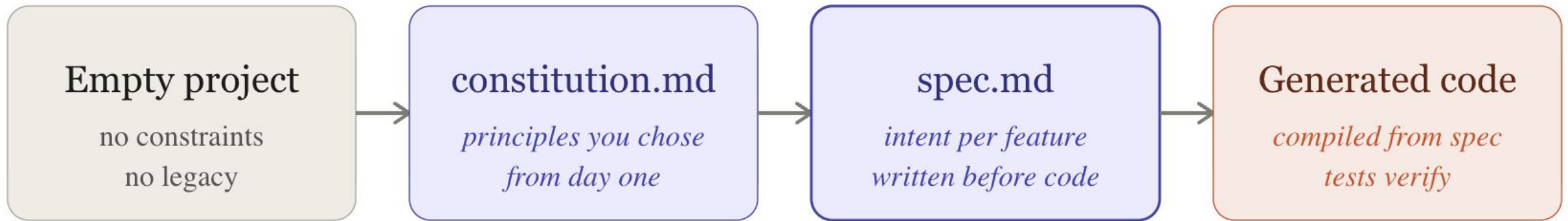
Scenario: Kafka consumer with a fraud-detection business rule.

```
/specs |- constitution.md <- immutable principles |-  
001-fraud-check/ |- spec.md <- the what |- plan.md <- the how  
|- tasks.md <- the breakdown
```

The Rule: "For each OrderPlaced event, if amount > 1000 then publish a FraudCheckRequested event."

SDD in a Greenfield

Starting from scratch — the spec is native from day one.

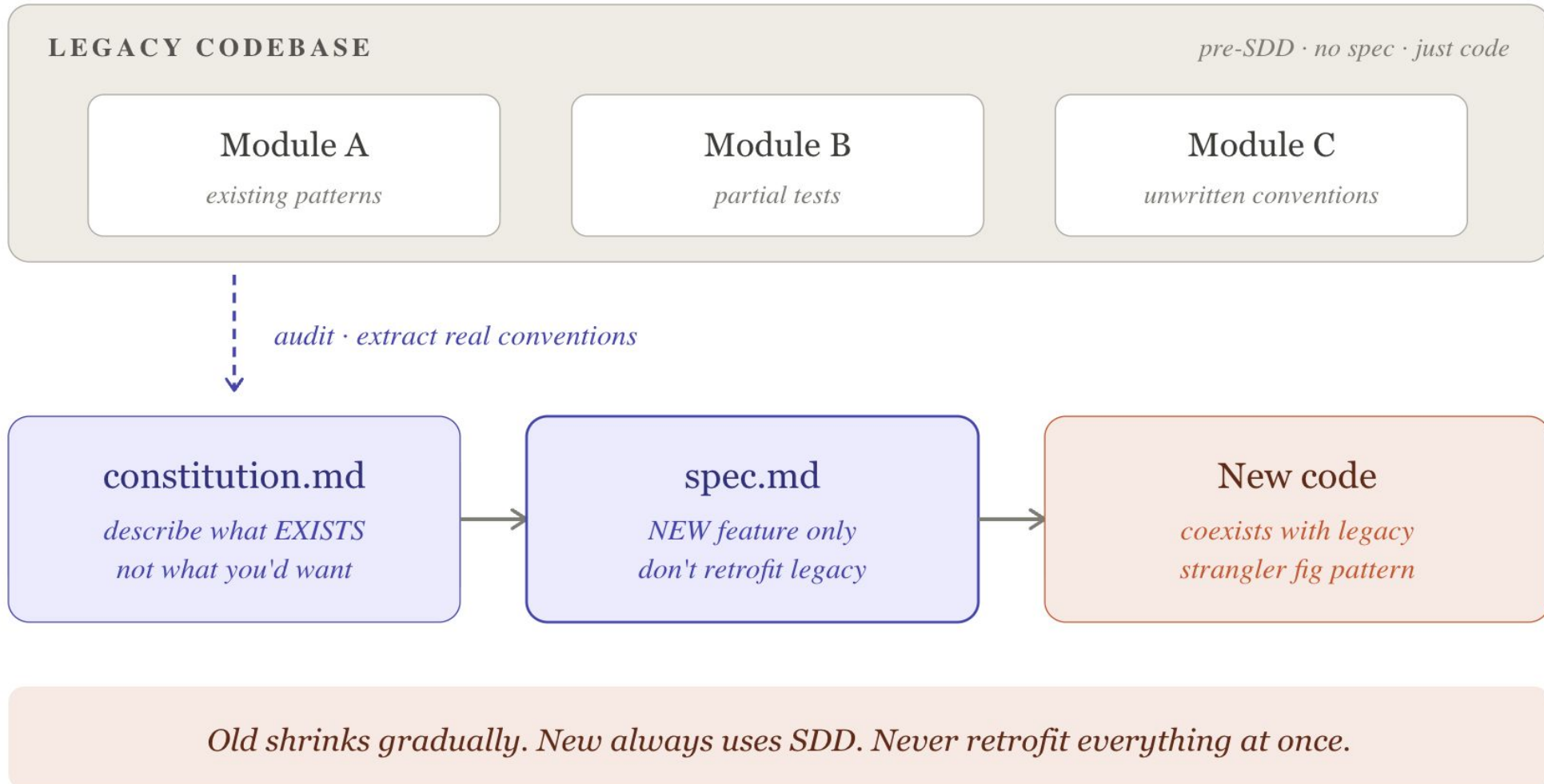


The spec is the source of truth from day one.

Best case for SDD. Your demo lives here.

SDD in a Brownfield

Adding SDD to an existing codebase — the real world for 95% of teams.



Anti-patterns to avoid

⚠ **Over-spec:** Specifying everything, including the obvious, leading to paralysis.

⚠ **Spec-then-forget:** Writing the spec but never maintaining it. Puts you back at square one.

⚠ **Blind trust:** Letting the AI generate without human review, causing you to inherit hallucinations.

⚠ **Context saturation:** Dumping everything into one giant .md file causing the agent to lose focus. *Fix: Split specs exactly like you split microservices.*

Anti-patterns to avoid

⚠ **Over-spec:** Specifying everything, including the obvious, leading to paralysis.

⚠ **Spec-then-forget:** Writing the spec but never maintaining it. Puts you back at square one.

⚠ **Blind trust:** Letting the AI generate without human review, causing you to inherit hallucinations.

⚠ **Context saturation:** Dumping everything into one giant .md file causing the agent to lose focus. *Fix: Split specs exactly like you split microservices.*

The right reflex: the spec is a dialogue, not a monologue.

Key Takeaways

- ✓ The spec **isn't dead** — it just changed roles.
- ✓ SDD is TDD/BDD in the age of AI: **the contract before the code**.
- ✓ You can **start tomorrow**: a /specs folder, a constitution.md, and an AI agent.
- ✓ AI amplifies your engineering discipline, it doesn't replace it.

Key Takeaways

- ✓ The spec **isn't dead** — it just changed roles.
- ✓ SDD is TDD/BDD in the age of AI: **the contract before the code**.
- ✓ You can **start tomorrow**: a /specs folder, a constitution.md, and an AI agent.
- ✓ AI amplifies your engineering discipline, it doesn't replace it.

"Stop patching the assembly. Update your intent, and let the compiler do its job."

Thank you !

Resources



<https://github.com/lelong1304/sdd>

